Sistema de Computación Ubicua

Comunicación Bidireccional entre Nodo Central y Nodos Sensores

Jose Mena P

Tecnologías Emergentes

26 de marzo de 2026

Contenido

1. Computación Ubicua
2. Arquitectura del Sistema
3. Nodo Sensor 1
4. Nodo Sensor 2
5. Nodo Central
6. Bot de Telegram
7. Interfaz Streamlit
8. Alertas y Umbrales
9. Ejecución y Verificación
10. Conclusiones

¿Qué es la Computación Ubicua?

La **computación ubicua** (ubiquitous computing) es el paradigma donde la tecnología está integrada en el entorno físico de forma transparente para el usuario.

Concepto clave

Los dispositivos computan y se comunican **en todo momento y lugar**, sin que el usuario deba interactuar directamente con ellos.

- Propuesto por Mark Weiser (Xerox PARC, 1991).
- Base del Internet de las Cosas (IoT).
- Los sensores recolectan datos; el nodo central decide y actua.
- Ejemplos: termostatos inteligentes, cámaras de seguridad, wearables.

Principios del Sistema Diseñado

Nuestro sistema aplica tres principios fundamentales de la computación ubicua:

Percepción

Los nodos sensores miden temperatura, humedad y movimiento del entorno físico.

Comunicación

Los nodos intercambian información con el nodo central mediante el protocolo HTTP.

Acción

El nodo central envía comandos para controlar actuadores (LEDs y motores).

Característica diferenciadora

La comunicación es **bidireccional**: los nodos no solo envían datos, también reciben y ejecutan órdenes del nodo central.

Diagrama General del Sistema

Usuario (Telegram / Streamlit)	
↕ comandos / alertas	
NODO CENTRAL nodo_central.py Monitoreo + Bot Telegram + Menú consola	
↕ HTTP GET /datos	↕ HTTP POST /comando
NODO SENSOR 1 localhost:5001 Temperatura + Humedad LED + Motor	NODO SENSOR 2 localhost:5002 Movimiento + Presencia LED

- Toda la comunicación usa **HTTP + JSON**.
- No requiere librerías externas: solo la biblioteca estándar de Python.
- Cada nodo corre en su propio proceso (terminal independiente).

Archivos del Proyecto

Archivo	Puerto	Función
`config.py`	—	Token Telegram, URLs, umbrales
`nodo_sensor_1.py`	5001	Servidor HTTP: temperatura, LED, motor
`nodo_sensor_2.py`	5002	Servidor HTTP: movimiento, LED
`nodo_central.py`	—	Monitoreo, alertas, bot Telegram
`interfaz.py`	8501	Panel Streamlit (visualización)

Dependencias

`pip install streamlit` (el resto usa solo la biblioteca estándar de Python 3)

Nodo Sensor 1 — Sensores Simulados

El Nodo 1 simula dos sensores físicos mediante funciones que generan valores aleatorios con comportamiento realista.

```python
def leer_temperatura():
    global _temperatura_base
    # Fluctuacion lenta del ambiente
    _temperatura_base += random.uniform(-0.3, 0.3)
    _temperatura_base = max(10.0, min(45.0, _temperatura_base))
    # Pico esporadico (5% probabilidad) para disparar alertas
    if random.random() < 0.05:
        return round(_temperatura_base + random.uniform(10, 15), 1)
    return round(_temperatura_base + random.uniform(-0.5, 0.5), 1)

def leer_humedad():
    return round(random.uniform(40.0, 80.0), 1)
```

Nodo Sensor 1 — Servidor HTTP

El nodo implementa un servidor HTTP con dos endpoints usando `http.server` de la biblioteca estándar de Python.

```python
class ManejadorNodo1(BaseHTTPRequestHandler):

    def do_GET(self):            # GET /datos
        if self.path == "/datos":
            respuesta = {
                "temperatura": leer_temperatura(),
                "humedad":     leer_humedad(),
                "led":         estado["led"],
                "motor":       estado["motor"],
            }
            self._responder_json(200, respuesta)

    def do_POST(self):           # POST /comando
        cuerpo  = json.loads(self.rfile.read(...))
        comando = cuerpo.get("comando", "").upper()
        if   comando == "LED_ON":    estado["led"]   = True
        elif comando == "LED_OFF":   estado["led"]   = False
        elif comando == "MOTOR_ON":  estado["motor"] = True
        elif comando == "MOTOR_OFF": estado["motor"] = False
```

Nodo Sensor 2 — Sensor de Movimiento

El Nodo 2 simula un sensor PIR (Passive Infrared) que detecta movimiento. La probabilidad varía según la hora del día para simular actividad real.

```python
def leer_movimiento():
    hora_actual = datetime.now().hour
    # Mayor probabilidad en horario diurno
    if 8 <= hora_actual <= 18:
        probabilidad = 0.25     # 25% durante el dia
    else:
        probabilidad = 0.10     # 10% en horario nocturno

    detectado = random.random() < probabilidad
    if detectado:
        _detecciones_movimiento += 1
    return detectado
```

Actuador

El Nodo 2 tiene un **LED de alerta** que el nodo central enciende automáticamente cuando detecta movimiento fuera del horario permitido.

Nodo Central — Arquitectura de Hilos

El nodo central ejecuta **tres tareas en paralelo** usando `threading`:

Hilo 1
Monitoreo

Cada 5 segundos consulta ambos nodos, verifica umbrales y registra fallos.

Hilo 2
Bot Telegram

Hace long polling a la API de Telegram y responde comandos del usuario.

Hilo Principal
Menú consola

Menú interactivo en terminal para enviar comandos manualmente.

```
hilo_monitoreo = threading.Thread(target=ciclo_monitoreo, daemon=True)
hilo_telegram  = threading.Thread(target=ciclo_bot_telegram, daemon=True)
hilo_monitoreo.start()
hilo_telegram.start()
menu_consola()   # hilo principal (bloquea hasta que el usuario salga)
```

Nodo Central — Ciclo de Monitoreo

```python
def ciclo_monitoreo():
    while True:
        # --- Nodo 1 ---
        datos1 = consultar_nodo(config.NODO_1_URL, "Nodo1")
        if datos1:
            nodo_recuperado("nodo_1", "Nodo1")
            verificar_temperatura(datos1)      # alerta si fuera de umbral
        else:
            verificar_nodo_caido("nodo_1", "Nodo1")  # alerta si sin respuesta

        # --- Nodo 2 ---
        datos2 = consultar_nodo(config.NODO_2_URL, "Nodo2")
        if datos2:
            nodo_recuperado("nodo_2", "Nodo2")
            verificar_movimiento(datos2)       # alerta si movimiento nocturno
        else:
            verificar_nodo_caido("nodo_2", "Nodo2")

        time.sleep(config.INTERVALO_POLLING_SEG)   # esperar 5 segundos
```

Bot de Telegram — Configuración

Pasos para crear el bot:

1. Buscar `@BotFather` en Telegram
2. Ejecutar `/newbot` y asignar nombre y username
3. Copiar el **token** generado (formato: `123456789:AABBcc...`)
4. Enviar un mensaje al bot y visitar: `api.telegram.org/bot{TOKEN}/getUpdates`
5. Anotar el campo ïd" dentro de çhat" (**Chat ID**)
6. Pegar ambos valores en `config.py`

config.py

```
TELEGRAM_TOKEN =
  "8689495203:AAFrKj..."

TELEGRAM_CHAT_ID =
  "933445452"
```

Bot de Telegram — Long Polling

El bot utiliza la técnica de **long polling** para recibir mensajes sin necesidad de un servidor público (no requiere webhook).

```python
def ciclo_bot_telegram():
    # Saltar mensajes previos al inicio del programa
    pendientes = obtener_actualizaciones_telegram(offset=0)
    offset = pendientes[-1]["update_id"] + 1 if pendientes else 0

    while True:
        actualizaciones = obtener_actualizaciones_telegram(offset)
        for update in actualizaciones:
            offset   = update["update_id"] + 1
            texto    = update["message"]["text"]
            chat_id  = str(update["message"]["chat"]["id"])
            respuesta = procesar_comando_telegram(texto)
            # Responder al usuario
            _llamar_api_telegram("sendMessage", {
                "chat_id": chat_id,
                "text":    respuesta,
            })
```

Bot de Telegram — Comandos Disponibles

Comando	Acción
`/ayuda`	Lista todos los comandos disponibles
`/estado`	Muestra temperatura, movimiento y actuadores
`/led1_on`	Enciende el LED del Nodo 1
`/led1_off`	Apaga el LED del Nodo 1
`/motor_on`	Enciende el motor del Nodo 1
`/motor_off`	Apaga el motor del Nodo 1
`/led2_on`	Enciende el LED del Nodo 2
`/led2_off`	Apaga el LED del Nodo 2

Alertas automáticas (sin comando del usuario)

El bot también envía mensajes proactivos cuando detecta condiciones anormales (temperatura, movimiento nocturno, nodo caído).

Interfaz Streamlit — Panel de Control

La interfaz `interfaz.py` provee un panel visual que:

- Consulta los nodos cada **3 segundos** (`st.rerun()`)
- Muestra el estado de cada actuador con indicadores de color:
  - • **Verde** = LED Nodo 1 encendido
  - • **Azul** = Motor encendido
  - • **Amarillo** = LED Nodo 2 encendido
  - • **Negro** = apagado
- Incluye botones *Encender* / *Apagar* para cada actuador
- Muestra alertas cuando hay movimiento detectado

Ejecutar

```
streamlit run interfaz.py
```

Se abre en `http://localhost:8501`

Interfaz Streamlit — Lógica de Auto-refresco

```python
# Consultar datos de ambos nodos en cada ciclo
datos1 = consultar_nodo(config.NODO_1_URL)
datos2 = consultar_nodo(config.NODO_2_URL)

# Mostrar indicador visual del LED
led1_on = datos1["led"]
color_led1 = "verde" if led1_on else "negro"
st.markdown(f"### {'Encendido' if led1_on else 'Apagado'}")

# Botones de control
if st.button("Encender LED"):
    enviar_comando(config.NODO_1_URL, "LED_ON")
    st.rerun()        # refrescar inmediatamente

# Auto-refresh automatico cada 3 segundos
time.sleep(3)
st.rerun()
```

Lógica de Alertas — Temperatura

```python
# Umbrales en config.py
TEMP_MAX_C = 30.0     # Alerta si temperatura SUPERA este valor
TEMP_MIN_C = 15.0     # Alerta si temperatura es INFERIOR a este valor

def verificar_temperatura(datos_nodo1):
    temp = datos_nodo1.get("temperatura")
    if temp > config.TEMP_MAX_C and not _alerta_temp_enviada["alta"]:
        msg = (f"ALERTA TEMPERATURA ALTA\n"
               f"Temperatura: {temp} grados C\n"
               f"Umbral maximo: {config.TEMP_MAX_C} grados C")
        enviar_mensaje_telegram(msg)
        _alerta_temp_enviada["alta"] = True
    elif temp <= config.TEMP_MAX_C:
        _alerta_temp_enviada["alta"] = False    # Resetear alerta
```

Mecanismo anti-spam

La alerta se envía **una sola vez** cuando se supera el umbral. Solo se vuelve a enviar si la temperatura regresa al rango normal y vuelve a subir.

Lógica de Alertas — Movimiento y Nodo Caído

Movimiento nocturno

```python
# Horario de alerta (config.py)
HORA_ALERTA_INICIO = 22
HORA_ALERTA_FIN    = 6

def verificar_movimiento(datos):
    if not datos.get("movimiento"):
        return
    hora = datetime.now().hour
    fuera = (hora >= 22) or (hora < 6)
    if fuera:
        enviar_mensaje_telegram(
          "ALERTA MOVIMIENTO "
          "NOCTURNO")
```

Nodo sin respuesta

```python
MAX_SIN_RESPUESTA = 3

def verificar_nodo_caido(
        nombre_clave,
        nombre_display):
    _fallos[nombre_clave] += 1
    fallos = _fallos[nombre_clave]
    if fallos == MAX_SIN_RESPUESTA:
        enviar_mensaje_telegram(
          f"ALERTA NODO SIN "
          f"RESPUESTA: "
          f"{nombre_display}\n"
          f"Intentos: {fallos}")
```

Resumen de Umbrales Configurados

Condición	**Umbral**	**Parámetro**	**Alerta**
Temperatura alta	$> 30{,}0\,°C$	`TEMP_MAX_C`	Sí, única vez
Temperatura baja	$< 15{,}0\,°C$	`TEMP_MIN_C`	Sí, única vez
Movimiento nocturno	22:00 – 06:00	`HORA_ALERTA_*`	Sí, por evento
Nodo sin respuesta	3 fallos seguidos	`MAX_SIN_RESPUESTA`	Sí, al llegar a 3

Todos los umbrales se modifican en `config.py` sin tocar el código principal.

Ejecución del Sistema

El sistema requiere **3 terminales** (más una opcional para Streamlit):

Terminal 1 — Nodo Sensor 1

```
cd "...SistemaUbicuo"&& python nodo_sensor_1.py
```

Terminal 2 — Nodo Sensor 2

```
cd "...SistemaUbicuo"&& python nodo_sensor_2.py
```

Terminal 3 — Nodo Central

```
cd "...SistemaUbicuo"&& python nodo_central.py
```

Terminal 4 — Interfaz Streamlit (opcional)

```
cd "...SistemaUbicuo"&& streamlit run interfaz.py
```

Verificación del Sistema

Componente	Cómo verificar	Resultado esperado
Nodo 1 activo	`GET localhost:5001/datos`	JSON con temperatura y LED
Nodo 2 activo	`GET localhost:5002/datos`	JSON con movimiento
Comando HTTP	`POST /comando {LED_ON}`	`{"ok": true}`
Bot Telegram	Escribir `/estado` al bot	Respuesta con datos
Alerta Telegram	Esperar pico de temperatura	Mensaje automático
Streamlit	`localhost:8501`	Panel con botones

Conclusiones

- Se implementó un sistema de **computación ubicua** completamente funcional usando solo Python y la biblioteca estándar.
- La comunicación es **bidireccional**: el nodo central consulta datos *y* envía comandos a los nodos sensores.
- La integración con **Telegram** permite monitoreo y control remoto en tiempo real desde cualquier dispositivo.
- El sistema detecta y notifica automáticamente **tres tipos de alertas**: temperatura fuera de rango, movimiento nocturno y nodo caído.
- La interfaz **Streamlit** proporciona una visualización clara del estado de los actuadores con control por clic.
- La arquitectura modular permite escalar fácilmente agregando más nodos o sensores con mínimos cambios en `config.py`.

¿Preguntas?

Sistema de Computación Ubicua

Jose Mena P — Tecnologías Emergentes